



内部任务市场的结算守恒：重复劳动拒付 与拜占庭罚没的机制设计

UDOS 推演引擎 v7.5.0 · 房文新 · 2026-09-20

证据等级：UDOS 数字回 reports7/ 原始 JSON，按 verified / cpu-proto / cpu-proxy / unverified
标注；单 seed 试点写作“试点点估计”，投稿前需 ≥ 30 seed 复现。

Settlement Conservation in an Internal Task Market: Denying Duplicate Labor and Slashing Byzantine Agents

证据等级：全部数字来自 CPU 确定性模拟（`reports7/matrix_scale_demo.json`，`evidence_grade=cpu-proto`；单元测试数字来自 `tests7/test_v749_market.py` 与 `market.py` 代码，verified），为单 seed 试点估计；策略族 × 罚没率 × 验收命中率的全因子仿真（≥1000 轮/格）尚未跑，相关位置保留 [RESULT NEEDED]。范本对照定位：本文为《UDOS 写作与排版规范 v2》阶段 B 升级稿。实证章按”实验设置 → 对照（单元测试账本 vs 矩阵 clean/stressed）→ 敏感性（罚没率/验收命中率解析阈值）→ 效度威胁”展开；守恒不变量式(1)–(3)集中编号，clean/stressed 两档对照表照 NSA 范本”同预算双档对照”的形式。

中文摘要（结构化四段式）

【背景与问题】 当多智能体系统从几十个 Agent 扩张到百万级，任务分配与贡献结算就从工程细节变成了治理问题。没有显式结算规则的 Agent 系统里，资源消耗是”暗账”：谁做了重复工作、谁拿了不该拿的报酬、总预算花光还是浪费，都不可见。账本的第一性要求不是”激励最优”，而是”账算得平”。

【方法】 本文报告 UDOS v7.4.9 的内部任务市场（`udos7/topology/market.py`）：一个”质量门槛 + 性价比授标 + 仅对验收通过的唯一完成者付费 + 拜占庭罚没”的结算机制。我们把结算规则写成优先级短路，并把”账算得平”形式化为一条守恒不变量，由 `conservation_check()` 以浮点容差强制。

【证据与结果】 核心守恒不变量是 `balance_sum = total_paid - slashed` 且 `total_paid ≤ total_budget`。混合 3 任务单元测试实测：预算 24、已付 16、罚没 1、余额和 15 (=16-1)、holds 8、`conserved=true`；矩阵端到端集成（24 工单、3 故障节点）clean/stressed 两档均 `conserved=true`、24/24 按验收结算、返工不产生重复支付。10 项单元测试覆盖授标、门槛、负载兜底、唯一付费、重复不付、罚没负余额、守恒、不超预算。

【贡献】 (1) 证明守恒是定理（由结算代码结构保证），而激励相容是经验近似（依赖验收命中率）；(2) 给出”结算规则 × 行为类型 → 守恒/激励”的实测矩阵；(3) 诚实标注它是内部记账单位、无真实定价博弈、无 Sybil 防御、不做金融化主张。

关键词： 机制设计；任务市场；结算守恒；拜占庭容错；多智能体系统；激励相容

Abstract (Structured, Four Parts)

[Background & Problem] As multi-agent systems scale from dozens to millions of agents, task allocation and contribution settlement move from an engineering detail to a governance problem. Without explicit settlement rules, resource consumption is a “dark ledger”; the first requirement of a ledger is not “optimal incentives” but “the books balance.”

[Method] This paper reports the internal task market of UDOS v7.4.9 (`udos7/topology/market.py`): a settlement mechanism combining a quality floor, a value-for-money award rule, payment only to the unique accepted completer, and slashing of Byzantine results. We write the settlement rule as

a priority short-circuit and formalize “balanced books” as a conservation invariant enforced under floating-point tolerance.

[Evidence & Results] The core invariant is `balance_sum = total_paid - slashed` with `total_paid ≤ total_budget`. A mixed 3-task unit test measures: budget 24, paid 16, slashed 1, balance sum 15 (=16-1), holds 8, `conserved=true`; the end-to-end matrix integration (24 orders, 3 faulty nodes) is `conserved=true` in both clean and stressed runs, 24/24 settled by acceptance, retries produce no duplicate payment. Ten unit tests cover award, floor, load tie-break, unique payment, duplicate denial, slashing, conservation, and never-exceed-budget.

[Contribution] (1) Conservation is a theorem (guaranteed by settlement code structure), whereas incentive compatibility is an empirical approximation (depends on acceptance hit rate); (2) we provide a measured “settlement rule × behavior type → conservation/incentive” matrix; (3) we honestly scope it as internal accounting units with no real pricing game, no Sybil defense, and no financialization claim.

Keywords: mechanism design; task market; settlement conservation; Byzantine fault tolerance; multi-agent systems; incentive compatibility

1 引言

多智能体编排正在从“几个 Agent 协作”走向“成百上千乃至百万级 Agent 矩阵”。当任务以工单形式在 Agent 之间流转时，立刻出现三个工程问题：谁有资格接这个任务？按什么规则授标？做完之后凭什么付钱、付多少？如果这些规则是隐式的、靠人肉协调，系统一放大就会出三件事：**重复劳动**（两个 Agent 做了同一件事，资源白白浪费）、**低质冲标**（最廉价但最偷懒的投标者总中标）、**谎报**（交一个根本不合格的结 果，却要求报酬）。

为什么这件事重要？因为在一个没有显式结算规则的 Agent 系统里，资源消耗是“暗账”：你不知道谁做了重复工作、谁拿了不该拿的报酬、总预算是被花光了还是被浪费了。当系统只有 10 个 Agent，人肉协调还能顶上；到 1000 个、100 万个 Agent，没有账本就无法治理。账本的第一性要求不是“激励最优”，而是“账算得平”——钱不凭空产生、不凭空消失，每一笔支出都能追溯到一个验收通过的工单。本文把这个最基本的工程要求形式化为守恒不变量，并证明它在代码结构上恒成立。

本文面对的就是这个结算治理问题。UDOS 推演引擎 v7.4.9 的 `udos7/topology/market.py` 给出了一个极简但自治的内部市场：任务先定价（`deposit`），Agent 投标（`Bid`：成本、自评质量、负载），拍卖按“质量门槛 + 性价比 + 负载兜底”授标（`TaskAuction.award`），完成后 QA 验收（`settle(accepted=...)`），只有验收通过的唯一完成者才拿钱；重复提交、验收不通过、拜占庭结果一律不付，拜占庭还要从保证金里扣（`slash`）。整个账本满足一个守恒不变量。

把这个机制拆开看，它其实回答了四个朴素的问题。**谁能接？**——质量门槛 $q_i \geq f$ 。**给谁？**——合格者里性价比 q_i/c_i 最高、负载最低者。**做完凭什么给？**——QA 验收通过才给。**给多少、怎么扣？**——给全额 reward，重复/拒绝/拜占庭给 0，拜占庭额外扣保证金。这四问听起来像常识，但把它们写成代码、写成测试、写成守恒断言，才是工程。本文的全部贡献，就是把这四问从“常识”变成“可执行、可测试、账算得平”的规则。

需要先声明边界。这是一个**内部记账单位**的市场，不是真实加密货币经济：没有真实定价博弈、没有 Sybil 防御、没有跨账户加密结算、不与任何通证/金融化挂钩。它回答的问题是”在一个封闭的、可审计的 Agent 系统内部，如何用记账规则让资源既不凭空产生、也不凭空消失”，而不是”如何设计一个开放的、防女巫的经济协议”。凡涉及外部”代币经济”“Agent 货币”的产业叙事，均为 unverified 报道口径，本文不与之互证。

本文的具体贡献：

1. **规则与不变量**：用机制设计语言写出支付规则，证明守恒是一个定理（由结算代码结构保证），而激励相容是经验近似（依赖验收命中率）。
2. **可复现测试**：10 项单元测试 + 矩阵端到端集成，给出”结算规则 $\times$ 行为类型 $\rightarrow$ 守恒/激励”的实测矩阵。
3. **诚实边界**：明确哪些是定理（守恒）、哪些是经验（IC 依赖验收命中率），不把工程原型夸大为机制设计理论贡献。

这三点贡献里，第三点可能最被低估。在 Agent 系统叙事普遍夸大的当下，一份诚实标注”这是内部记账单位、无 Sybil 防御、IC 只是经验近似”的论文，比一份自称”去中心化自主经济”的白皮书更有长期价值——因为前者知道自己的边界，后者不知道。

选择把”守恒”而非”最优拍卖”作为本文主角，是一个有意的价值取向。在机制设计教科书里，VCG 那种”说真话最优”的优雅结论很吸引人，但它在工程上需要公共偏好聚合、维数灾难、策略证明的计算成本；而一个 300 行的 `market.py`，用优先级短路和一个守恒不变量，已经把”钱不凭空多出来”这件最基本的事做扎实了。对一个百万级 Agent 系统来说，”账算得平”比”拍卖最优”更紧迫——账不平，再优的拍卖也建立在流沙上。

边界声明：全部数字来自 CPU 确定性模拟（`reports7/matrix_scale_demo.json`，`evidence_grade=cpu-proto`；单元测试数字来自 `tests7/test_v749_market.py` 与 `market.py` 代码），为单 seed 试点点估计；策略族 $\times$ 罚没率 $\times$ 验收命中率的全因子仿真（ ≥ 1000 轮/格）尚未跑，相关位置保留 `[RESULT NEEDED]`。

2 相关工作

任务分配与拍卖在多智能体系统、分布式系统与机制设计里都有成熟脉络。**概念**上，分布式任务分配最早由合同网协议形式化：节点通过”招标—投标—授标”三段协商把任务发包给最有能力者（Smith, 1980），这正是 UDOS `TaskAuction` 投标/授标的历史源头。**机制**上，Vickrey 指出二价密封拍卖让说真话成为占优策略（Vickrey, 1961），其推广 VCG 在理想假设下同时实现激励相容与帕累托有效；但 Ausubel 与 Milgrom 在综述中冷静地指出，VCG”理论上优雅、实践中孤独”——它计算困难、易被串谋、且不满足预算平衡，因此现实市场设计中罕有大规模部署（Ausubel & Milgrom, 2006）。现代市场设计进一步把视角拓展到组合拍卖与采购场景，讨论电力、频谱、采购等”买方向多个卖方买服务”的授标规则如何在真实约束下工作（Milgrom, 2019）。**证据**上，拜占庭容错在分布式系统侧有成熟的状态机复制传统：PBFT 在异步网络下容忍 $f < n/3$ 拜占庭节点，且实现开销仅比未复制系统大约 3%（Castro & Liskov, 1999）；而”罚没”作为一种拜占庭经济惩罚，在区块链/staking 文献里另有大量讨论 [CITATION NEEDED: slashing Proof of Stake]。**与 UDOS 差异**：上述工作多在开放经济或共识层，不解决”一个封闭内部市场里结算账本怎么算平”；女巫抵抗（Sybil resistance）在开放经济下更是硬问题 [CITATION NEEDED: Sybil attack Douceur]。

与本文最接近的是”多智能体任务分配带支付”这一支 [CITATION NEEDED: multi-agent task allocation payment; market-based multi-robot task allocation; peer prediction mechanism proper scoring]。这类工作通常用拍卖把任务配给最低成本或最高效用的 Agent。我们的差异点不在拍卖理论，而在**结算账本**的工程化守恒：大多数拍卖工作关心”谁中标”，本文关心”中标之后怎么付钱、怎么保证钱不凭空多出来或少掉、怎么让重复劳动和谎报在账面上无利可图”。换言之，我们把 VCG 关心的”真话占优”退一步，换成工程上可审计的”验收通过才付、唯一完成者才付、拜占庭必扣”——这是对 Ausubel-Milgrom 所指出的 VCG 实践缺陷的一种务实回应。

我们的定位是**机制工程 + 仿真参考实现**，不主张 VCG 式的理论最优。理由是：在封闭内部市场里，完全 IC 的复杂机制（VCG 需公共偏好聚合、计算贵）不划算；一个简单的”质量门槛 + 性价比 + 验收付费 + 罚没”在工程上可审计、可测试，已经足够抑制重复劳动与谎报。理论深度不足时，我们诚实地把它定位为经验论文。

还需要把本文与”通证经济/Agent 货币”这类流行叙事划清界限。后者常主张”给 Agent 发币、让市场自动调节”，但那是开放经济问题，需要 Sybil 防御、价格发现、外部信任锚，远超本文范围。这两者之间存在一个本文要填补的空白：大多数拍卖工作关心”谁中标”，很少有把”中标后怎么算账、怎么保证守恒、怎么让重复劳动和谎报无利可图”写成可测试不变量的工程参考实现。学术界对 VCG 的激励相容有漂亮定理，但对”一个 300 行、可被 pytest 覆盖、账算得平的内部账本”这种工程问题关注不足。本文不追求 VCG 式优雅，而是提供一个可直接读、可直接跑、可直接扩展的结算参考实现，让后续工作在它上面做全因子策略仿真，而不是从零写账本。

需要承认，这种工程取向的代价是理论深度有限：我们不证明我们的拍卖是最优的、不证明我们的罚没机制是防共谋的、不证明在一般效用域下 IC 成立。本文的所有结论都限定在”内部记账单位、oracle 验收、单线程顺序结算”这一具体工程配置内。把它当成一个机制设计理论贡献是误读；把它当成一个可复现的工程基准，才是本文的定位。

3 问题定义与假设

3.1 记号与规则

一个任务 t 有预算 B_t (`deposit` 注入)。Agent i 投标 (c_i, q_i, ℓ_i) ：成本 c_i 、自评质量 $q_i \in [0, 1]$ 、负载 ℓ_i 。拍卖门槛 f 。授标规则：

$$\text{text}\{\text{eligible}\} = \{i: q_i \geq f\}, \text{text}\{\text{winner}\} = \arg\max_{i \in \text{text}\{\text{eligible}\}} \left(\frac{q_i}{\max(c_i, 10^{-9})} \right), -\ell_i.$$

即先过质量门槛，再按性价比 q/c 授标，负载作次序兜底（同性价比选负载低者）。

结算规则 (`settle`) 按优先级短路：

1. 若任务已付过 (`task_id` in `paid_tasks`) $\rightarrow$ 付 0，理由 `already_paid`；
2. 若标为重复劳动 (`duplicate=True`) $\rightarrow$ 付 0，理由 `duplicate_work`；
3. 若验收不通过 (`accepted=False`) $\rightarrow$ 付 0，理由 `rejected`，并从该 Agent 余额扣 s （罚没）；
4. 否则（验收通过且未付过、非重复） $\rightarrow$ 付 B_t 给该 Agent，记入 `paid_tasks`。

3.2 可证伪假设

- **H1（重复劳动支付为 0）**：唯一付费约束使任何重复/重复提交的支付为 0。可证伪：
`test_only_accepted_unique_work_paid / test_duplicate_work_flag_not_paid` 任一失败。
- **H2（罚没使拜占庭期望收益为负）**：在给定罚没率 s 与验收命中率 p 下，拜占庭策略期望收益 < 0 。可证伪：当 s 足够小、验收命中率足够高时，拜占庭期望收益 ≥ 0 。
- **H3（质量门槛防低价低质）**：门槛 f 过滤掉 $q_i < f$ 的投标。可证伪：
`test_quality_floor_filters_cheap_low_quality` 失败。
- **H4（预算守恒）**：`total_paid  $\leq$  total_budget` 且 `balance_sum = total_paid - slashed`。可证伪：`conservation_check().conserved` 为 `False`。

还有一个隐含假设值得显式化：**H5（验收是 oracle）**。当前 QA 把”结果是否合格”判定为已知的布尔值，验收命中率 $p = 1$ 。这意味着本文的 IC 结论是在”谎报必被抓”的理想验收下成立的；一旦验收有漏网，H2 就要按 4.5 节公式重算。我们不把这个假设藏起来，而是把它列为 H5，让读者一眼看出 IC 的成立条件。

4 方法与系统设计

4.1 拍卖：性价比授标

`TaskAuction.award` 的 `eligible` 过滤与性价比排序是两行代码的事，但设计意图值得说清。为什么用 q/c 而非单纯最低成本？因为单纯最低成本会激励”低价低质”——谁报得最低谁中标，质量自然劣化。为什么加质量门槛 f 而非纯性价比？因为性价比 q/c 在 q 很小时仍可能很高（ c 极小），门槛先把”根本不合格”的投标者挡在门外，再在合格者里选性价比最优。负载 ℓ_i 作为次序兜底：两个投标性价比相同时，选当前负载低的，避免把所有任务压到同一个 Agent。

表 2 单元测试里 `test_auction_picks_best_quality_cost_ratio` 直接演示了这一点：三个投标者 $a(\text{cost}=8, q=0.8)$ 、 $b(\text{cost}=4, q=0.8)$ 、 $c(\text{cost}=3, q=0.2)$ 。 a 和 b 质量相同 0.8， b 成本低，性价比 b 胜 a ； c 虽成本最低但质量 0.2 被视为低质。最终 b 中标。这说明 q/c 同时惩罚了”太贵”和”太差”，而不是只看价格。

4.2 结算账本：优先级短路

`ContributionLedger.settle` 的优先级短路是这个机制的核心。它先问”这个任务付过没有”，再问”是不是重复劳动”，再问”验收过没过”，最后才付钱。每一层短路都对应一种策略行为的防线：

- **已付检查防**”一个任务被两个 Agent 都交了、付两次”；
- **重复标记防**”同一个 Agent 反复交同一件事刷工作量”；
- **验收检查防**”交个不合格结果也拿钱”；
- **罚没**在验收失败时额外扣保证金，提高谎报的期望成本。

4.3 守恒不变量

`conservation_check()` 计算：

$$\text{total_budget} = \sum_i B_i, \quad \text{balance_sum} = \sum_i \text{bal}_i,$$

$\text{holds} = \text{total_budget} - \text{total_paid},$

$\text{conserved} = (\text{total_paid} \leq \text{total_budget} + \epsilon) \wedge |\text{balance_sum} - (\text{total_paid} - \text{slashed})| < \epsilon, \quad \epsilon = 10^{-9}.$

其中 $\epsilon = 10^{-9}$ 容忍浮点误差。这个不变量是**定理而非经验**：因为 `settle` 只在验收通过时把 B_t 从预算池转入某个 Agent 余额（`total_paid += B_t, balances[agent] += B_t`），罚没只从余额扣（`balances[agent] -= s, slashed += s`），没有任何代码路径凭空创造余额，所以账本恒等在算术上恒成立。真正需要测试的是”规则是否如设计般触发”，而非”算术是否守恒”。

值得展开的是”为什么守恒值得专门证明”。在工程实践里，账本 bug 最常见的形式不是”钱算错了”，而是”钱不知道去哪了”：一笔付出去了但没记账、一笔罚没扣了余额但没记 `slashed`、一个任务付了两次。这些 bug 在小规模系统里靠人肉对账能发现，在百万级系统里就是黑账。把 `balance_sum = total_paid - slashed` 写成一行可执行的 `conservation_check()`，并在每个集成测试结束时调用一次，就是把”对账”自动化成 CI 门禁：任何一笔钱去向不明，`conserved` 立刻变 `False`。这种”把会计恒等式变成测试断言”的做法，比事后审计便宜得多。

4.4 定理与经验的边界

需要严格区分两类结论。**守恒是定理**：只要 `settle` 不被绕过、不出现浮点漂移，`conserved` 恒真。**激励相容 (IC) 是经验近似**：拜占庭策略是否负收益，取决于”验收命中率 p ”与”罚没率 s ”的组合——如果验收太松（ p 低）或罚没太轻（ s 小），拜占庭期望收益可能转正。本文当前只测了一个固定配置（ $s = 1.0$ 、验收严格），全因子阈值网格待补（[RESULT NEEDED]）。

4.5 个体理性与激励相容的成立条件

把这套规则放到机制设计语言里，可以更精确地说清它承诺了什么、没承诺什么。

个体理性 (IR)：诚实 Agent 参与市场的期望效用非负。因为验收通过才付钱、且付的是任务全额 reward，诚实完成者的净收益为正（reward 减去其真实成本，只要 $\text{reward} > \text{cost}$ 就值得做）；不参与则收益为 0。因此在” $\text{reward} \geq \text{真实成本}$ ”的任务定价前提下，诚实 Agent 有动机参与。但这依赖任务定价本身合理——如果 reward 定得低于真实成本，连诚实 Agent 都不愿接，IR 就被破坏。本文把定价合理假设为外部前提，不在市场机制内解决。

激励相容 (IC)：诚实投标是否优于谎报？在本机制里，IC 是**部分的、经验的**。重复劳动被唯一付费约束堵死（无论怎么重复，第二个及以后的提交都付 0），这部分 IC 是结构性的。但”低质冲标”能否被抑制，取决于质量门槛 f 与验收命中率 p ：如果自评质量 q_i 可以谎报（Agent 自评 0.9 实际交付 0.3），门槛就被绕过，只能靠验收兜底；验收命中率 p 越低，低质交付漏网概率越高。拜占庭罚没 s 进一步提高谎报的期望成本，但 s 必须大于”谎报被验收通过的期望收益”才有威慑力。

可以把拜占庭期望效用写成：

$$E[u_{\text{byz}}] = (1-p)R - ps,$$

其中 R 是谎报被漏验时的非法收益、 p 是验收命中率、 s 是罚没。当 $(1-p)R < ps$ 即 $s > R(1-p)/p$ 时，拜占庭期望收益为负。这就是”罚没率盈亏平衡点”的解析形式，也是 [RESULT NEEDED] 全因子仿真要验证的阈值。本文不在无 R 、 p 实测的情况下手算这个阈值。

5 实验设置

按 v2 规范，实证章按”实验设置 → 对照（单元测试账本 vs 矩阵 clean/stressed）→ 敏感性（罚没率/验收命中率解析阈值）→ 效度威胁（第 8 章）“展开。

5.1 单元测试协议

`tests7/test_v749_market.py` 含 10 个测试，全部确定性、CPU、无随机：

1. 性价比授标（同质低价者胜）；
2. 质量门槛过滤低价低质；
3. 无合格投标者无中标；
4. 负载打破平局；
5. 仅验收通过的唯一完成者付费（重复完成不付）；
6. 重复劳动标记不付；
7. 拜占庭结果不付且可罚没；
8. 守恒不变量（混合 3 任务）；
9. 支付永不超预算；
10. 被拒工作被记录。

5.2 端到端集成

`scripts7/matrix_demo.py` 跑 24 工单的 Agent 矩阵，clean（无故障）与 stressed（注入 3 个故障节点）两档，市场结算由 `udos7/topology/matrix.py` 调用。输出 `reports7/matrix_scale_demo.json`（`evidence_grade=cpu-proto`）。硬件：CPU、确定性模拟，Agent 非真实 LLM。

需要说明，矩阵集成里的市场预算是 240（24 工单 × 10/工单），而单元测试的混合 3 任务预算是 24（ $T1=10+T2=8+T3=6$ ）。两者不是同一组数字：单元测试是最小可复现的守恒演示，矩阵集成是 24 工单的规模演示。clean/stressed 两档都把 240 全部付完（`total_paid=240`），因为所有工单最终都被验收通过；单元测试则故意保留一个验收失败的 T2，让 `holds=8>0`，以演示”未付预算挂账”这一非平凡情形。如果所有工单都顺利验收，`holds` 恒为 0，守恒退化为 `total_paid=total_budget`——这种平凡情形无法检验 `holds` 字段，因此单元测试特意构造了 T2 失败。

6 结果

6.1 单元测试守恒：混合 3 任务

表 1 复现 `test_conservation_invariant` 的账本。三个任务：T1=10（付给 a）、T2=8（验收拒绝、罚没 1.0）、T3=6（付给 c）。

表 1 混合 3 任务单元测试账本（verified, `test_conservation_invariant`）

量	值	说明
<code>total_budget</code>	24	$T1+T2+T3$

量	值	说明
total_paid	16	T1(10)+T3(6)
slashed	1	T2 罚没
holds	8	T2 未付预算仍在池
balance_sum	15	=16 已付 -1 罚没
conserved	true	不变量成立

表 1 注：证据等级 verified（单元测试硬编码断言）。式(4) 双侧条件均满足： $total_paid=16 \leq total_budget=24+\varepsilon$ ； $balance_sum=15=(total_paid\ 16)-(slashed\ 1)$ ，误差 $<\varepsilon$ 。holds=8 即式(3)，为 T2 未付预算挂账。

`balance_sum=15` 精确等于 `total_paid(16) - slashed(1)`，验证了守恒不变量。T2 验收失败未付，其 8 预算留在 `holds`（池子）里，不消失、也不凭空转给别人。图 1 把这个账本与矩阵 clean/stressed 两档并列。

这里需要解释 `holds=8` 的会计含义。总预算 24 = 已付 16 + 未付 holds 8。未付的 8 是 T2 被验收拒绝的那笔预算，它既没付给 a/b/c（那会造成超支或错付），也没从系统里消失（那会破坏守恒），而是”挂”在池子里等待重新分配——这正是 `holds` 字段的作用：追踪”还悬而未决、尚未流转给任何人”的预算。这种设计让账本永远闭合：预算发行总量 = 已付给 Agent 的 + 被罚没扣回的 + 仍在池子的。任何时刻查账，`total_budget` 都能拆成这三项，没有”黑账”。



图 1 三账本守恒对照。左列：单元测试混合 3 任务（budget 24 = paid 16 + slashed 1 + holds 8；balance_sum 15）；中列：矩阵 clean 档（budget 240 = paid 240 + holds 0）；右列：矩阵 stressed 档（budget 240 = paid 240 + holds 0）。每档均 `conserved=true`。证据等级：单元测试 verified、矩阵 cpu-proto。数据出处 `reports7/matrix_scale_demo.json` + `tests7/test_v749_market.py`。

6.2 结算规则 × 行为类型

表 2（图 2）是”结算规则 × Agent 行为 → 支付结果”的决策矩阵。这是 H1/H2/H3 的直接对应。

表 2 结算规则 × 行为类型 (verified, 来自 market.py settle 逻辑)

Agent 行为	是否付费	理由	是否罚没
诚实提交、验收通过	付全额 reward	accepted	否
重复提交同任务	付 0	already_paid / duplicate_work	否
低质提交、验收拒绝	付 0	rejected	可选 (slash>0)
拜占庭结果、验收拒绝	付 0	rejected	是 (扣保证金)
无合格投标者	无人中标	—	—

表 2 注：证据等级 verified (`settle` 优先级短路逻辑 + 10 项单元测试)。读法：逐行”该行为落入哪条短路→付多少→是否罚没”，即每种策略行为的账面结果。拜占庭”期望收益为负”的严格结论需全因子仿真 (式(5))，当前为 `[RESULT NEEDED]`。

图 P11-2 结算规则 × Agent 行为 → 支付 (来源 market.py settle 优先级短路)

诚实提交 验收通过	付全额 reward
重复提交 同任务	付 0 (already_paid)
低质提交 验收拒绝	付 0 (rejected)
拜占庭结果 验收拒绝	付 0 + 罚没保证金

图 2 结算规则 × 行为类型决策矩阵。横轴为 Agent 行为 (诚实/重复/低质/拜占庭/无合格投标)，纵轴为结算短路层级 (已付检查→重复标记→验收检查→付费)，汇向支付结果 (全额/0/无人中标) 与是否罚没。证据等级 verified；拜占庭期望收益符号为 `[RESULT NEEDED]`。

H1 成立：诚实提交付全额，其余三类策略行为支付均为 0。H3 成立：质量门槛在授标阶段就把 $q < f$ 挡掉 (`test_quality_floor`)，无需等到验收。H2 部分成立：拜占庭支付为 0 且被罚没，但”期望收益为负”的严格结论需要全因子仿真——当前只在 $s = 1.0$ 、验收严格的单点上观察到负余额，`[RESULT NEEDED]`：罚没率 {0,0.5,1,2} × 验收命中率 {0.7,0.9,1.0} 网格]。

具体到 `test_rejected_byzantine_work_unpaid_and_slashable`：拜占庭 Agent z 初始余额 3.0，验收拒绝后 `settle(accepted=False, slash=2.0)`，z 余额降到 1.0、`slashed=2.0`、`total_paid=0`。这意味着 z 不仅没拿到报酬，还倒赔了 2.0 保证金——在这个配置下，谎报的账面成本是确定的。但这是一次确定性演示，不是期望收益统计；z 会不会谎报，取决于它对”自己被验收抓到的概率 p”的估计，而这个 p 在当前 oracle 验收下恒为 1 (只要交了坏结果就一定被抓)，所以 z 必亏。一旦验收有漏网 ($p < 1$)，结论就要按 4.5 节的公式重算。

另外两个值得记录的授标细节：`test_load_breaks_tie` 验证同性价比时选负载低者（load=1 胜 load=5），避免任务扎堆；`test_no_eligible_bid_no_winner` 验证无人合格时不硬授标（winner 保持 None），这是”宁可空着也不硬派”的保守设计，比”随便选一个”更安全。

6.3 端到端集成：clean vs stressed

表 3 与图 3 给出矩阵集成的两档对照。

表 3 矩阵端到端结算（cpu-proto, 24 工单；来源 `reports7/matrix_scale_demo.json`）

指标	clean	stressed (3 故障节点)
n_orders	24	24
accepted	24	24
qa_rounds	24	25
retries	0	5
isolated_agents	0	3
total_budget	240	240
total_paid	240	240
slashed	0	0
balance_sum	240	240
conserved	true	true
duplicate_work	0	0

表 3 注：证据等级 cpu-proto（单 seed 试点）。两档 total_paid=240=total_budget, 故 holds=0、balance_sum=240, 式(4) 恒成立。stressed 档注入故障节点 `spec-kinematics-0 / spec-spatial-1 / spec-dataflywheel-2`, 经熔断器隔离（isolated_agents=3）产生 5 次返工（retries=5、qa_rounds=25），但 `duplicate_work=0`——返工是重做同一工单，唯一付费约束保证不重复支付。stressed 档 `slashed=0` 是因为本跑故障走熔断路径而非市场罚没路径，不代表罚没机制缺失。



图 3 矩阵端到端 clean/stressed 双档对照。左 clean、右 stressed；上排为治理/结算指标（n_orders、accepted、retries、isolated_agents），下排为账本（budget/paid/slashed/balance_sum/conserved）。两档均 `conserved=true`、24/24 按验收结算。证据等级 cpu-proto（单 seed 试点）。数据出处 `reports7/matrix_scale_demo.json`。

两档都 24/24 按验收结算、`conserved=true`。stressed 档注入了 3 个故障节点（`spec-kinematics-0`、`spec-spatial-1`、`spec-dataflywheel-2`），系统通过熔断器把它们隔离（`isolated_agents=3`），产生 5 次返工（`retries=5`、`qa_rounds=25`），但返工不产生重复支付——`duplicate_work=0`，因为 `settle` 的唯一付费约束保证一个工单只付一次。值得注意：stressed 档 `slashed=0`，因为本集成的故障治理走的是熔断器隔离（circuit breaker）而非市场罚没；罚没机制本身在单元测试（表 1、表 2）里验证。这是两条独立的防线：市场罚没管”谎报但完成了”，熔断隔离管”节点坏了根本完不成”。

从治理指标看，clean 档 governance 全零（`state_loss=0`、`duplicate_work=0`、`no_closer=0`、`premature_completion=0`、`unbounded=0`、`trace_gaps=0`），`trace_verified=true`、`trace_len=48`；stressed 档同样 governance ok、`trace_verified=true`、`trace_len=50`。也就是说，即使注入 3 个故障节点、产生 5 次返工，治理审计没有发现任何重复劳动、状态丢失或 trace 断裂。这印证了”唯一付费约束”在端到端集成里也起作用：返工是重做同一个工单，不会因为重做而付第二次钱。

7 讨论

守恒为什么是定理而 IC 是经验。这一区分很重要，因为它决定了我们能承诺什么。守恒：只要代码路径不绕过 `settle`，算术恒等——这是可承诺的。IC：拜占庭是否负收益，取决于验收命中率与罚没率的组合——这只能在具体配置下测。本文当前的罚没配置（`s = 1.0` 在 T2 上）让拜占庭 Agent z 的余额从 3.0 降到 1.0（`test_rejected_byzantine_work_unpaid_and_slashable`），账面上是负收益；但若把 `s` 设为 0，拜占庭就只损失时间、不损失保证金，期望收益就可能转正。因此本文不承诺”在任意罚没率下拜占庭都负收益”，只承诺”在 `s = 1.0`、验收严格这一配置下观察到负余额”。

还有一个值得展开的设计选择：为什么罚没走保证金扣减，而不是直接没收全部余额？`settle` 的 `slash` 是一个可配参数（默认 0，调用方传入），这意味着罚没力度是一个策略旋钮，而非硬编码。在单元测试里我们用 `slash=1.0`（T2）和 `slash=2.0`（z）两种力度演示：前者是”轻微谎报”，后者是”明确拜占庭”。把力

度做成参数，是因为不同任务的危害程度不同——一个写错文档的任务，和一个伪造安全关键结果的任务，理应罚没不同倍数。本文不规定最优罚没率，那是第 4.5 节阈值网格要回答的问题。

还有一个设计哲学值得记录：这个机制**宁可空着也不错付**。`test_no_eligible_bid_no_winner` 明确，当所有投标者都过不了质量门槛时，拍卖返回 None、不硬派。这与“必须在 deadline 前把任务派出去”的激进调度相反。在一个内部市场里，硬派给一个不合格的投标者，最终只会产生返工和重复支付；宁可让任务暂时无人认领，等下一轮投标或升级给上级，也比错付强。这种“保守优先”的默认值，和 P5 的“宁可区间宽一点”、P6 的“宁可说不可辨识”是同一个工程伦理：在不确定时选择保守的那一侧。

8 局限与效度威胁（独立章）

T1 内部威胁（最主要）。单元测试是确定性单点，未做 ≥ 1000 轮策略仿真，IC 结论窄。**最可能的反例：**换罚没率 s 或验收命中率 p ，拜占庭期望收益转正。**本文明怎么防：**把式(5) 盈亏平衡点 $s > R(1-p)/p$ 写成解析阈值，全因子网格标 `[RESULT NEEDED]`。

T2 构造威胁：验收是 oracle。当前 QA 把“结果是否合格”判定为已知布尔值（验收命中率 $p = 1$ ），真实系统里验收本身可能被操纵——这是更深的治理问题，本文未解。

T3 外部威胁。CPU 原型、Agent 非真实 LLM，百万级规模只做了闭式外推，未在真实分布式系统跑。

T4 测量威胁。stressed 档 `slashed=0` 不代表罚没机制无效，只代表本跑没触发罚没路径（故障走了熔断隔离路径）。

T5 并发威胁。`ContributionLedger` 用普通 Python `dict` 记账，假设单线程顺序结算。真实分布式部署里，若两个 Agent 几乎同时提交同一工单结果，单线程 `dict` 可能竞态：两个 `settle` 都读到“未付过”然后都付款，守恒被破坏。当前靠调用方（`matrix.py`）单线程串行调用 `settle` 规避，而非账本本身并发安全。守恒在“顺序结算”下是定理，在“并发结算”下需要 linearizability 保证，后者本文未解。

T6 无 Sybil 防御。“唯一付费”靠 `paid_tasks` 的 `task`→`agent` 映射，假设一个 `task` 只有一个 `owner`；它不防 Sybil——若攻击者用多个身份投标并重复提交，`duplicate` 标记需上游正确识别”这是重复劳动”才触发。本文诚实声明：无 Sybil 防御、无身份押金、无加密结算。

9 资源闸门与适用边界

- 本文结论停在 cpu-proto（CPU 确定性原型）。全因子策略仿真（ ≥ 1000 轮/格）、真实 LLM Agent 投标、真实分布式账本均待补。
- 不与任何真实加密/通证经济挂钩，不做金融化主张。
- 适用边界：本文方法适用于“封闭、可审计、内部记账”的多智能体任务市场；不适用于开放经济、需 Sybil 防御、需真实货币结算的场景。
- 与 P1/P2/P3 的关系：P1 讲拓扑降复杂度、P2 讲 BFT 停止决策、P3 讲故障治理 RCT，本文讲治理闭环里的钱怎么结算——四者合起来才是一个完整的 Agent 矩阵治理栈。

具体而言，资源闸门对 P11 结论的限制体现在三处。其一，**策略仿真未跑：**当前只有确定性单元测试和端到端两档，没有“诚实/重复/低质冲标/拜占庭/合谋压价”五种策略在不同罚没率与验收命中率下的期望收益分布，H2 的 IC 结论因此只能停在单点。其二，**验收是 oracle：**当前 QA 规则明确、验收命中率恒为 1（坏结果必被抓），真实系统里验收本身可能被操纵或漏判，这是更深的治理问题。其三，**规模未实测：**百万级

Agent 只做了闭式外推（见 P1），结算账本在真实分布式并发下的一致性（比如两个 Agent 同时提交同一工单的竞态）需要分布式账本机制，本文的单线程 `dict` 实现不处理并发。

10 结论

我们在 UDOS v7.4.9 的内部任务市场上，给出一个”质量门槛 + 性价比授标 + 仅对验收通过的唯一完成者付费 + 拜占庭罚没”的结算机制。守恒不变量 `balance_sum = total_paid - slashed` 是定理，混合 3 任务与矩阵 `clean/stressed` 两档均实测 `conserved=true`；重复劳动与谎报在账面上支付为 0，10 项单元测试全过。诚实的部分：IC 是经验近似而非定理，罚没率/验收命中率的全因子阈值网格未跑；无 Sybil 防御、无真实定价博弈。我们把可承诺的（守恒）与只能近似的（IC）分开写，把未做完的用 `[RESULT NEEDED]` 标出。一个可审计的内部市场，首先是一个账算得平的市场。

后续工作有三条。第一，跑全因子策略仿真（策略族 {诚实,重复,低质冲标,拜占庭,合谋压价} × 罚没率 {0,0.5,1,2} × 验收命中率 {0.7,0.9,1.0}, ≥1000 轮/格），定位罚没率盈亏平衡点 $s^* = R(1-p)/p$ ，把 H2 从单点升级为可外推的阈值曲线。第二，把结算账本接到真实 LLM Agent 投标上，检验”自评质量 `q_i` 能否被诚实报告”——这是 IC 最脆弱的一环。第三，处理并发竞态：当前单线程 `dict` 实现不处理”两个 Agent 同时提交同一工单”的竞态，分布式部署需要 linearizability 的账本后端。三篇合读（P5 可信度 + P6 可辨识性 + P11 结算）共同指向一个工程哲学：诚实的系统不是什么都能做，而是知道自己什么不能做——结算上，就是知道哪笔钱该付、哪笔钱必须扣、哪笔钱宁可挂着也不能错付。

最后回到本文最朴素的主张：在多智能体系统走向百万级的过程中，“账算得平”是比”机制最优”更基础、也更稀缺的能力。本文用 300 行代码和 10 个测试证明了这一点。它不优雅、不复杂、不理论，甚至有点无聊——但一个不无聊、不出错、可审计的内部账本，正是支撑上层一切智能协作的地基。地基不需要漂亮，需要的是承重。愿这份 300 行代码的诚实账本，能成为后来者在百万级 Agent 系统上讨论结算问题时，一个可以直接读、直接改、直接跑的起点，而不是又一份没人能复现的白皮书。

参考文献

说明：本稿已引用的已核验文献字段照抄全卷《参考文献总库》（核验日期 2026-09-19）；其余为待核验检索式，不作已引用事实，投稿前须逐条经学术检索核验元数据与 DOI。

已核验引用 (✓ 总库 2026-09-19)

- Smith, R. G. (1980). The Contract Net Protocol: High-Level Communication and Task Control in a Distributed Problem Solver. IEEE Transactions on Computers, C-29(12). DOI: 10.1109/TC.1980.1675516. —— 招标—投标—授标协商，UDOS 任务分发与 `TaskAuction` 的历史源头。
- Vickrey, W. (1961). Counterspeculation, Auctions, and Competitive Sealed Tenders. Journal of Finance, 16(1). —— 二价拍卖=VCG 雏形，真话为占优策略。
- Ausubel, L. M., & Milgrom, P. (2006). The Lovely but Lonely Vickrey Auction. In Combinatorial Auctions (书章), MIT Press. —— VCG 理论美与实践缺陷（串谋/预算失衡/计算难），本文”务实退一步”的直接依据。

- Milgrom, P. (2019). Auction Market Design: Recent Innovations. Annual Review of Economics, 11. —— 组合拍卖/电力/频谱/采购的现代市场设计，procurement auction 范式参照。
- Castro, M., & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999 (USENIX). —— 异步下容忍 $f < n/3$ 拜占庭节点；本文拜占庭罚没的分布式系统对照。

待核验文献检索式与候选方向

- 合同网/任务分配：multi-robot task allocation market-based; multi-agent task allocation payment mechanism design。
- 拍卖理论：procurement reverse auction; peer prediction mechanism proper scoring rule (peer prediction 核心文献待补)。
- 拜占庭/罚没：slashing Proof of Stake economic penalty。
- 女巫抵抗：Sybil attack Douceur; Sybil resistance open economy。

附录 A 复现命令与测试索引

```
cd release-v5.4.4/udos-engine          # main, tag v7.5.0, commit 1a71270
pytest tests7/test_v749_market.py -q    # 10 项单元测试
python3 scripts7/matrix_demo.py         # 端到端集成 -> reports7/matrix_scale_demo.json
```

关键源码：udos7/topology/market.py (Bid / TaskAuction / ContributionLedger)、udos7/topology/matrix.py (集成)、tests7/test_v749_market.py。

复现顺序建议：先跑 `pytest tests7/test_v749_market.py -q` 看 10 个单元测试全绿（这是守恒与规则的最小验证），再跑 `matrix_demo.py` 看端到端两档 `conserved=true`。单元测试是确定性的、秒级完成；矩阵集成稍慢但仍是 CPU。任何一步 `conserved=false` 都说明账本被改坏了——这正是 `conservation_check()` 作为 CI 门禁的意义。

附录 B 证据台账

数字	来源	等级
total_paid=16/holds=8/ balance_sum=15/slashed=1	tests7/test_v749_market.py: test_conservation_invariant	verified (单元测试)
矩阵 clean/stressed conserved=true	matrix_scale_demo.json: clean/stressed.market	cpu-proto
24/24 accepted, retries 0/5, isolated 0/3	matrix_scale_demo.json	cpu-proto
10 项单元测试	tests7/test_v749_market.py	verified
外部代币经济数字	—	unverified

最后提醒：本文所有”达标”“通过”的措辞都限定在”CPU 确定性原型、单线程顺序结算、oracle 验收”这一具体口径内。它不是”机制设计在任意多智能体系统下都成立”的普适结论，而是”在这个 300 行 `market.py`、10 项 `pytest`、24 工单矩阵集成的具体系统上，结算守恒由代码结构保证”。超出这一口径的任何推广，都需要新的实验与新的证据。

[RESULT NEEDED]：罚没率 $\times$ 验收命中率全因子 ≥ 1000 轮仿真、各策略期望收益 CI、误罚率（诚实者被罚的概率）。[CITATION NEEDED]：第 2 节五处候选方向。

补充说明：表中”单元测试”数字（`total_paid=16` 等）来自 `test_conservation_invariant` 的硬编码断言，是确定性可复算的；”矩阵 `clean/stressed`”数字来自 `matrix_scale_demo.json`，是 `cpu-proto` 原型运行结果，两者证据等级不同，不应混用。`stressed` 档 `slashed=0` 不代表罚没机制缺失，而代表该跑的故障走熔断路径——这一区分已在正文 6.3 节如实说明。

附录 C 内审记录（投稿前五维自评，只看不改）

按 `doubao-academic-evaluator` 五维打分（0–10，`cpu-proto` 原型口径）：

1. 问题重要性：5。内部市场结算随 Agent 规模扩张确实重要，但本文定位工程参考实现，非理论突破。
2. 技术正确性：7。守恒不变量由代码结构保证、测试覆盖完整；扣分在 IC 仅单点、无全因子仿真。
3. 实验严谨度：5。10 单元测试 + 端到端两档是硬证据；但策略族仿真未跑，IC 结论窄。
4. 新颖性：5。机制组件经典（拍卖/罚没/账本），新颖性在守恒测试工程化与诚实边界声明。
5. 可复现性：9。脚本、测试、报告 JSON 齐全，CPU 可复跑。

致命缺陷检查：守恒 vs IC 的边界已诚实区分；无伪造 p 值/CI/DOI；无 Sybil 防御已声明。结论：当前为可投稿的经验论文初稿，投稿前必须补全因子策略仿真以把 H2 从单点升级为阈值网格，否则只能投 workshop/工程 track。

作者拟用声明

- 目标期刊/会议：候选 AAMAS / AAMAS workshop；偏工程可投 ICWS、JSS。若理论深化可投 EC。分区/IF [待核]。
- 预注册计划：守恒不变量为硬约束（已在代码中）；罚没率 $\times$ 验收命中率网格的因子设计与停止规则投稿前登记。
- 数据与代码可用性：Apache-2.0；tag `v7.5.0`、commit `1a71270`；报告 `reports7/matrix_scale_demo.json`、`tests7/test_v749_market.py`。
- AI 使用声明：作者主导机制设计与结论，AI 协助结构化写作、图表与语言润色；数字经作者对照源码与测试回源，AI 未产生实验数据。